Name – Sunny Raja Prasad

Roll number – 211078

Ans1(a) –

```
Function multiplyMatrices(matrix1, matrix2):
   n = size(matrix1)
   result = initialize an n x n matrix with zeros

   for i = 0 to n-1:
      for j = 0 to n-1:
         for k = 0 to n-1:
            result[i][j] += matrix1[i][k] * matrix2[k][j]

   return result

Function matrixExponentiation(baseMatrix, exponent):
n = size(baseMatrix)
   result = initialize an n x n identity matrix

   currentBase = baseMatrix

   while exponent > 0:
      if exponent is odd:
         result = multiplyMatrices(result, currentBase)
      currentBase = multiplyMatrices(currentBase, currentBase)
```

```
        exponent = exponent / 2

    return result

Function calculateFinalWealth(initialWealth, months):
    n = size(initialWealth)

    wealthDistribution = initialize an n x n matrix with zeros
    for i = 0 to n-1:
        if 2*i + 1 < n:
        // Distribute wealth equally to child companies
            wealthDistribution[i][i] = 0.5
            wealthDistribution[i][2*i + 1] = 0.5

    finalDistribution = matrixExponentiation(wealthDistribution, months)

    currentWealth = initialWealth

    finalWealth = initialize a vector of size n with zeros
    for i = 0 to n-1:
        for j = 0 to n-1:
            finalWealth[i] += finalDistribution[i][j] * currentWealth[j]

    return finalWealth
```

Ans 1(b) - The provided algorithm calculates the final wealth of each company after m months in a hierarchical binary tree structure. Let's analyze its time complexity and correctness.

Time Complexity:

1. Multiplying Two Matrices (multiplyMatrices function):

- Multiplying two n x n matrices takes O(n^3) time.
- This operation is performed inside two nested loops, iterating over n rows and n columns, resulting in O(n^3) complexity.

2. Matrix Exponentiation (matrixExponentiation function):

- The matrix exponentiation process involves repeated matrix multiplications.
- The exponentiation process runs in O(log(m)) time, where m is the input exponent.
- In each iteration, the size of the matrices being multiplied is reduced by half.

3. Constructing the Wealth Distribution Matrix:

- Constructing the wealth distribution matrix (wealthDistribution) takes O(n) time as it involves iterating through the companies in the tree.
- Each company is assigned a distribution value, which takes constant time.

4. Final Wealth Calculation:

- Calculating the final wealth involves matrix-vector multiplication, which takes O(n^2) time.
- This operation is performed once, so it doesn't depend on the exponent m.

Overall, the time complexity of the algorithm is dominated by the matrix exponentiation step, which is O(n^3 * log(m)) due to the repeated matrix multiplications. The other steps have lower time complexities.

Correctness:

The algorithm appears to be correct based on the provided pseudocode.

- It constructs the wealth distribution matrix correctly based on the problem statement.
- It performs matrix exponentiation efficiently to compute the final distribution of wealth.
- It multiplies the final distribution matrix with the initial wealth vector to calculate the final wealth of each company.

Ans 1(c) -

```
Function calculateFinalWealth(root_wealth, m):
    final_wealth = root_wealth << m  // Use left shift operator to double the wealth

    return final_wealth
```

Algorithm Description:

The algorithm aims to calculate the final wealth of a single company (root) in a binary tree after m months. It takes advantage of the fact that the wealth of the company doubles every month.

1. Input: The initial wealth of the root company (root_wealth) and the number of months (m) for which we want to calculate the final wealth.

2. Initialization: Initialize a variable final_wealth to store the final wealth.

3. Calculate Final Wealth:

- Use the left shift operator (<<) to double the root_wealth by m bits.
- Assign the result to final_wealth.

4. Output: Return final_wealth as the final wealth of the root company after m months.

Ans 2(a) –

```
Function minCostRoomAllocation(capacities, n, p, C):
    left = 0
    currentSum = 0
    minCost = Infinity

    for right = 0 to n-1:
        currentSum += capacities[right]

        while currentSum >= p:
            minCost = min(minCost, C * (right – left + 1))
            currentSum -= capacities[left]
            left += 1

    return  minCost

// Example usage
capacities = array of room capacities
n = number of rooms
p = minimum total capacity required
C = cost of booking a room

minCostRoomAllocation(capacities, n, p, C)
```

Ans 2(b) -

```
Function isBeautifulAllocation(capacities, n, K, length):
   gcd = capacities[0]

   # Calculate the GCD for the first 'length' elements
   For i from 1 to length-1:
      gcd = GCD(gcd, capacities[i])

   if gcd >= K:
      return True
   else:
      return False

Function maxBeautifulAllocation(capacities, n, K):
   left = 1
   right = n
   maxRooms = 0
    # Binary search to find the maximum contiguous rooms
   While left <= right:
      mid = (left + right) / 2

      # Check if there's a beautiful allocation with 'mid' rooms
      if isBeautifulAllocation(capacities, n, K, mid):
         maxRooms = mid
```

```
        left = mid + 1
    else:
        right = mid - 1

return maxRooms
```

Algorithm Description:

1. isBeautifulAllocation checks whether a contiguous allocation of 'length' rooms has a GCD greater than or equal to K using the blackbox GCD algorithm. It calculates the GCD for the first 'length' elements of the capacities array.

2. maxBeautifulAllocation performs a binary search to find the maximum number of contiguous rooms that satisfy the beauty constraint (GCD >= K). It initializes a variable maxRooms to keep track of the maximum rooms found.

3. In each iteration of the binary search, it checks if there's a beautiful allocation with 'mid' rooms. If yes, it updates maxRooms and searches the right half. Otherwise, it searches the left half.

4. The binary search continues until the left pointer is less than or equal to the right pointer.

Time Complexity Analysis:

- The binary search performs O(log n) iterations.
- For each iteration, the isBeautifulAllocation function takes O(n) time to calculate the GCD for 'mid' rooms.
- Overall, the time complexity is O(n log n) due to the binary search, which is well within the required O(n log n) time complexity.

Ans 2(c) - **Correctness:**

The goal of this function is to find the minimum cost of booking a consecutive sequence of rooms whose total capacity is at least P. The correctness of the algorithm relies on two key observations:

1. Greedy Selection: The algorithm uses a greedy approach by continuously extending the right boundary of the current sequence of rooms (window) until the total capacity is at least 'P'. This ensures that it always selects the minimum number of rooms necessary to meet the capacity requirement.

2. Optimal Substructure: When the algorithm finds a window with total capacity greater than or equal to 'P', it calculates the cost of booking these rooms. By considering each valid window and selecting the one with the minimum cost, the algorithm ensures that it chooses an optimal combination of rooms for the given problem.

By iterating through the rooms from left to right and adjusting the window as needed, the algorithm guarantees that it identifies the minimum cost room allocation that meets the capacity requirement.

Time Complexity:

Let's analyze the time complexity of the minCostRoomAllocation function:

• The function uses a single pass through the array of room capacities, iterating from right = 0 to right = n-1. This loop has a time complexity of O(n).

• Inside the loop, there's another while loop that extends the window from the left (by incrementing left) if the total capacity is greater than or equal to P. In the worst case, where the entire array needs to be considered, the while loop would have a total cost of O(n).

Hence, the overall time complexity of the function is O(n), making it efficient for finding the minimum cost room allocation. It processes each room once, and the while loop ensures that each room's capacity is only accounted for once or not at all.

In conclusion, the minCostRoomAllocation function is both correct and efficient, with a time complexity of O(n).

Ans 3(a) –

```
Function findSwappedNodesAndAncestors(root):
    prev = null
    first = null
    second = null

    Function inorderTraversal(node):
        if node == null:
            return
        inorderTraversal(node.left)
        if prev != null and node.value < prev.value:
            if first == null:
                first = prev
                second = node
            else:
                second = node
                prev = node
        inorderTraversal(node.right)

    inorderTraversal(root)
    SwapValues(first, second)
    commonAncestors = FindCommonAncestors(root, first, second)
```

```
    return first, second, commonAncestors


Function FindCommonAncestors(root, first, second):
    commonAncestors = []


    While root != null:
        if root.value > first.value and root.value > second.value:
            commonAncestors.append(root)
            root = root.left
        else if root.value < first.value and root.value < second.value:
            commonAncestors.append(root)
            root = root.right
            else:
            // Common ancestor found, add it to the list if not already added
            if root not in commonAncestors:
                commonAncestors.append(root)
            Break


    return commonAncestors
```

Ans 3(b) –

```
Function countRearrangedNodes(root, originalValues, G):

    Function inorderTraversal(node, values):

      if not node:

          return []


      left_subtree = inorderTraversal(node.left, values)

      current_value = node.val

      right_subtree = inorderTraversal(node.right, values)


      return left_subtree + [current_value] + right_subtree


    Function isSameTree(node1, node2):

      if not node1 and not node2:   return True

      if not node1 or not node2:      return False

      if node1.val != node2.val:       return False

      return isSameTree(node1.left, node2.left) and isSameTree(node1.right, node2.right)


    # In-order traversal of the original tree to get its values

    original_values_inorder = inorderTraversal(root)


    # Sort the original values for comparison

     sorted_original_values = sorted(originalValues)
```

```
 # Check for rearranged nodes by comparing with sorted values
rearranged_nodes_count = 0
rearranged_nodes = []

for i = 0 to length(original_values_inorder) - 1:
    if original_values_inorder[i] != sorted_original_values[i]:
        rearranged_nodes_count += 1
        rearranged_nodes.append(original_values_inorder[i])
# Check if G is less than nlog(n)
if G < n * log2(n):
    return rearranged_nodes_count, rearranged_nodes

# Check if the entire tree is rearranged
if isSameTree(root, original_tree):
    return n, original_values_inorder

return rearranged_nodes_count, rearranged_nodes
```

Algorithm Description:

This code defines a function countRearrangedNodes that traverses the BST to count and identify rearranged nodes based on the original values. It checks two cases: when G is smaller than nlog(n) and when G is greater than or equal to nlog(n). Depending on these cases, it returns the count of rearranged nodes and their values.

- If G is smaller than nlog(n), it assumes a few nodes are rearranged.
- If G is larger, it checks if the entire tree is rearranged.

This algorithm has a time complexity of O(min(G + n, nlog(n))) as required, where G is the upper bound constant, and n is the number of nodes in the BST.

Ans 4(a) –

```
function findRotatedArrayPivot(arr):
    n = length(arr)
    start = 0
    end = n - 1

    while start <= end:
        mid = (start + end) / 2

        if arr[mid] >= arr[start]:
            # We are in the first part before the pivot.
            if arr[mid] <= arr[mid + 1]:
                start = mid + 1
            else:
                # Pivot found
                return mid + 1
        else:
            # We are in the second part after the pivot.
            if arr[mid] >= arr[mid - 1]:
                end = mid - 1
             else:
                # Pivot found
                return mid
    return 0
```

Ans 4(b) - The time complexity of the **findRotatedArrayPivot** algorithm is O(log n), where n is the length of the input array **arr**. Here's a step-by-step analysis:

1. The algorithm uses a binary search approach to find the pivot element, which reduces the search space by half in each iteration.
2. In each iteration of the **while** loop, the algorithm calculates the middle index **mid** as the average of **start** and **end**. This step takes O(1) time.
3. The algorithm then compares **arr[mid]** with **arr[start]** to determine whether it's in the first part (before the pivot) or the second part (after the pivot) of the rotated array. These comparisons take O(1) time.
4. Depending on the comparison results, the algorithm updates the **start** or **end** indices to narrow down the search range.
5. The binary search continues until it finds the pivot element or determines that the array is not rotated. In the worst case, this takes O(log n) iterations.
6. Finally, the algorithm returns the index of the pivot element or 0 if the array is not rotated.

Overall, the time complexity of this algorithm is dominated by the binary search step, making it O(log n) in the worst case. It efficiently finds the pivot element in a rotated array.

Ans 5 –

```
Function countGoodSubarrays(arr):
    n = length(arr)
    total_count = 0

    For i from 0 to n-1:
        left = 0
        right = min(i, n-i-1)
        count_left = 0

        While left <= right:
            mid = (left + right) / 2
            If isSubarrayGood(arr, i - mid, i + mid) is True:
                left = mid + 1
                count_left = mid
            Else:
                right = mid - 1
                 total_count += (1 + count_left)

        left = 0
        right = min(i+1, n-i-1)
        count_right = 0

        While left <= right:
            mid = (left + right) / 2
            If isSubarrayGood(arr, i - mid + 1, i + mid) is True:
```

```
            count_right = mid
            left = mid + 1
        Else:
            right = mid - 1

    total_count += count_right

return total_count
```

Algorithm Description:

In this algorithm, we iterate through each index in the given array and determine the length of the longest palindromic subarray that can be created using binary search. We do this for both odd and even-length palindromes.

1. For each index in the array:
    - We check if a palindrome of odd length (2*mid+1) can be formed using binary search.
    - Similarly, we check if a palindrome of even length (2*mid) can be formed using binary search.
2. In the case of an odd-length palindrome, the current element (ith element) becomes the middle element, and for even-length palindromes, any one of the two halves can be the center.

Time Complexity Analysis:

The time complexity of this algorithm is O(n * log(n)) for each element in the array since we perform binary search for both odd and even-length palindromes. Therefore, the overall time complexity is O(n * 2 * log(n)), which simplifies to O(n * log(n)).